R3.07
SQL dans un langage de programmation

Chapitre 6
Patron Repository et Spring Data JPA



Selena Lamari et Chouki Tibermacine

BUT 2 Informatique — IUT de Vannes — Université Bretagne Sud

Plan du cours

1. Du DAO au Repository

2. Spring Data JPA

3. Transactions et concurrence avec Spring

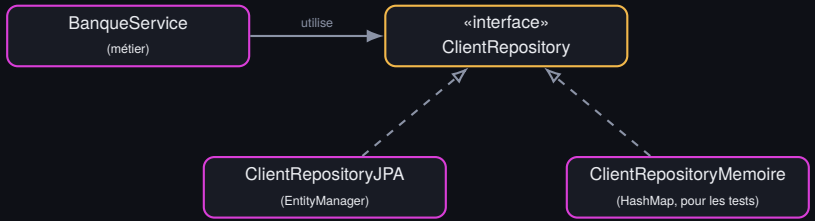
4. Performances

Où en est-on ?

Étape	Ce qu'on écrit	Ce qui est généré
JDBC (ch. 1)	SQL, `PreparedStatement`, mapping à la main	—
DAO (ch. 5)	une classe par table, SQL ou JPQL	—
JPA (ch. 4)	entités annotées, JPQL	le SQL CRUD
Repository (TP5)	une interface **métier** + son implémentation JPA	le SQL CRUD
Spring Data	l'interface seulement	le SQL **et** l'implémentation

- DAO : vocabulaire **technique**, une classe par table (`insertClient`, `updateClient`)
- Repository : une **collection d'objets métier** dont on ignore le stockage (`save`, `findById`, `findComptesDuClient`)

Le Repository inverse la dépendance



- Le service ne connaît que l'**interface** : ni SQL, ni JPA, ni `EntityManager`
- On peut changer de technique de persistance sans toucher au métier
- On peut **tester le métier sans base de données**

Tester le métier sans base

```
class ClientRepositoryMemoire implements ClientRepository {
  private final Map<Integer, Client> donnees = new HashMap<>();
  public void save(Client c)                 { donnees.put(c.getId(), c); }
  public Optional<Client> findById(int id)   { return Optional.ofNullable(donnees.get(id));
      }
  public List<Client> findAll()              { return new ArrayList<>(donnees.values()); }
  public List<Compte> findComptesByClient(int id) {
    return findById(id).map(Client::getComptes).orElse(List.of());
  }
}

@Test void soldeTotalDUnClient() {
  ClientRepository repo = new ClientRepositoryMemoire();
  repo.save(clientAvecDeuxComptes(1, 100, 50));
  assertEquals(150, new BanqueService(repo).soldeTotal(1));   // aucun Docker, aucun SQL
}
```

Ce qui se répète d'un Repository à l'autre

```
public Optional<Client> findById(int id) {
  return Optional.ofNullable(
      em.find(Client.class, id));
}
public List<Client> findByVille(String v) {
  return em.createQuery(
      "SELECT c FROM Client c WHERE c.ville
     = :v",
      Client.class)
    .setParameter("v", v).getResultList();
}
```

- Le même code pour `Client`, `Compte`, `Operation`...
- La requête se déduit du **nom** de la méthode

⇒ Un framework peut l'écrire à notre place : Spring Data

Dans le TP5, vous écrivez cette implémentation à la main pour comprendre ce que Spring Data génère.

Plan du cours

1. Du DAO au Repository

2. Spring Data JPA

3. Transactions et concurrence avec Spring

4. Performances

Mise en place (Spring Boot)

`pom.xml`

```
<dependency>
  <groupId>org.springframework.boot</groupId
    >
  <artifactId>spring-boot-starter-data-jpa</
    artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

`application.properties`

```
spring.datasource.url=jdbc:postgresql://
    localhost:5432/banque_db
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
```

- Plus de `persistence.xml` ni d'`EntityManagerFactory` à créer
- Pool de connexions (HikariCP) fourni
- Les entités JPA du chapitre 4 sont réutilisées telles quelles

Déclarer un Repository : une interface suffit

```
public interface ClientRepository extends JpaRepository<Client, Integer> {
  // rien a ecrire pour le CRUD
}

@Service
public class BanqueService {
  private final ClientRepository clients;
  public BanqueService(ClientRepository clients) {   // injection par constructeur
    this.clients = clients;
  }
}
```

- Au démarrage, Spring **génère une classe** qui implémente l'interface et l'injecte
- Méthodes héritées : `save`, `saveAll`, `findById`, `findAll`, `existsById`, `count`, `deleteById`, `findAll(Pageable)`, `findAll(Sort)`...
- `Client` et `Integer` : type de l'entité et type de sa clé primaire

Requêtes dérivées du nom de la méthode

```
List<Client>  findByVille(String ville);
List<Client>  findByNomClientStartingWithOrderByNomClientAsc(String prefixe);
long          countByVille(String ville);
boolean       existsByNomClient(String nom);
List<Compte>  findByClientNumClient(int numClient);          // dans CompteRepository
List<Compte>  findBySoldeGreaterThanEqual(BigDecimal min);
```

SQL réellement généré par Hibernate (extraits du journal) :

```
select ... from client c1_0 where c1_0.ville=?
select ... from client c1_0 where c1_0.nom_client like ? escape '\' order by c1_0.nom_client
select count(c1_0.num_client) from client c1_0 where c1_0.ville=?
select ... from compte c1_0 left join client c2_0 on c2_0.num_client=c1_0.num_client where c2_0.
    num_client=?
```

- Mots-clés : `And`, `Or`, `Between`, `LessThan`, `Like`, `In`, `IsNull`, `OrderBy`, `First10`…
- Un nom mal formé ou une propriété inconnue : erreur **au démarrage**, pas à l'exécution

Quand le nom devient trop long : `@Query`

```java
@Query("SELECT c FROM Client c WHERE size(c.comptes) >= :n")          // JPQL
List<Client> avecAuMoins(@Param("n") int n);

@Query(value = "SELECT * FROM client WHERE ville = ?1 LIMIT 5",      // SQL natif
       nativeQuery = true)
List<Client> cinqClientsDe(String ville);

@Modifying                                                           // UPDATE / DELETE
@Query("UPDATE Compte c SET c.solde = c.solde - :m WHERE c.numCompte = :num AND c.solde >=
    :m")
int debiter(@Param("num") int num, @Param("m") BigDecimal m);         // nb de lignes
    modifiees
```

- `@Modifying` : mise à jour relative du chapitre 2 (`solde = solde - ?`), exécutée dans une transaction
- Attention : une requête `@Modifying` **contourne** le contexte de persistance et le compteur `@Version` (vérifié : la version du compte reste à 0)

Projections : ne charger que l'utile (DTO, chapitre 5)

```
public interface ClientResume {          // projection : seulement des getters
  String getNomClient();
  String getVille();
}
List<ClientResume> findByVilleOrderByNomClient(String ville);
```

```
select c1_0.nom_client,c1_0.ville from client c1_0 where c1_0.ville=? order by 1
```

- Seules les colonnes demandées sont lues (chapitre 3 : « pas de `SELECT *` »)
- Variante : un `record` Java comme type de retour
- Le service renvoie ces projections ou des DTO, jamais les entités, à la couche présentation

Pagination : `Page` (OFFSET) ou `Window` (par clé)

Par numéro de page

```
Page<Client> findByVille(String v, Pageable p);
Page<Client> page = repo.findByVille("Vannes", PageRequest.of(1, 20, Sort.by("nomClient")));
```

```
select ... where c1_0.ville=? order by c1_0.nom_client offset ? rows fetch first ? rows only
```

Par clé (Spring Data 3.1+)

```
Window<Client> findFirst20ByVilleOrderByNumClient(String v, ScrollPosition pos);
Window<Client> w = repo.findFirst20ByVilleOrderByNumClient("Vannes", ScrollPosition.keyset());
w = repo.findFirst20ByVilleOrderByNumClient("Vannes", w.positionAt(w.size() - 1));  // page suivante
```

```
select ... where c1_0.ville=? and (c1_0.num_client>?) order by c1_0.num_client fetch first ? rows only
```

`Page` est pratique (numéro de page, nombre total) mais repose sur `OFFSET`, lent sur les pages lointaines (chapitre 3). `Window` produit la pagination par clé.

Plan du cours

1. Du DAO au Repository

2. Spring Data JPA

3. Transactions et concurrence avec Spring

4. Performances

`@Transactional` sur la couche service

```java
@Service
public class BanqueService {
  @Transactional                                          // BEGIN ... COMMIT / ROLLBACK
  public void virer(int src, int dst, BigDecimal montant) {
    if (comptes.debiter(src, montant) == 0) throw new SoldeInsuffisantException(src);
    comptes.crediter(dst, montant);
    operations.save(new Operation(src, dst, montant));
  }
  @Transactional(readOnly = true)
  public BigDecimal soldeTotal(int numClient) { ... }
}
```

- Remplace le schéma `setAutoCommit(false)` / `commit()` / `rollback()` du chapitre 2
- Rollback par défaut sur les exceptions **non vérifiées** (`RuntimeException`) seulement ; sinon : `@Transactional(rollbackFor = Exception.class)`
- Niveau d'isolation : `@Transactional(isolation = Isolation.REPEATABLE_READ)`
- Sans effet sur une méthode `private` et lors d'un appel interne (`this.virer(...)`) : la transaction est posée par un **proxy** autour du bean

Verrou optimiste : `@Version`

```java
@Entity
public class Compte {
  @Id private Integer numCompte;
  private BigDecimal solde;
  @Version private int version;
  ...
}
```

```sql
update compte set num_client=?,solde=?,version=?
 where num_compte=? and version=?
```

Si la ligne a changé depuis la lecture :

```
ObjectOptimisticLockingFailureException:
Row was updated or deleted by another transaction
```

- C'est la « solution 3 » du chapitre 2, écrite par Hibernate
- À attraper dans le contrôleur : message « données modifiées entre-temps, rechargez » ou nouvelle tentative

Verrou pessimiste : `@Lock`

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT c FROM Compte c WHERE c.numCompte = :num")
Optional<Compte> findPourMaj(@Param("num") int num);
```

```
select ... from compte c1_0 where c1_0.num_compte=? for no key update
```

- « Solution 2 » du chapitre 2. Sur PostgreSQL, Hibernate produit `FOR NO KEY UPDATE`, une variante de `FOR UPDATE` qui ne bloque pas les insertions de lignes filles référençant ce compte
- Appelée hors transaction : `InvalidDataAccessApiUsageException: Query requires transaction be in progress`

Plan du cours

1. Du DAO au Repository

2. Spring Data JPA

3. Transactions et concurrence avec Spring

4. Performances

Le problème N+1 et `@EntityGraph`

```
for (Client c : repo.findByVille("Vannes"))
  System.out.println(c.getComptes().size());
```

```
select ... from client c1_0 where c1_0.ville=?
select ... from compte c1_0 where c1_0.num_client
    =?
select ... from compte c1_0 where c1_0.num_client
    =?
select ... from compte c1_0 where c1_0.num_client
    =?
```

1 + N requêtes

```
@EntityGraph(attributePaths = "comptes")
List<Client> findWithComptesByVille(String v
    );
```

```
select ... from client c1_0
  left join compte c2_0
    on c1_0.num_client=c2_0.num_client
 where c1_0.ville=?
```

1 requête

Toujours activer `spring.jpa.show-sql` en développement et **compter** les requêtes. Alternative : `JOIN FETCH` dans un `@Query`.

Quand revenir au SQL

- Requêtes analytiques complexes (fenêtres, CTE récursives) : `@Query(nativeQuery = true)`
- Traitements de masse : un seul `UPDATE` plutôt que charger 100 000 entités
- Logique exécutée dans la base (chapitre 7) :

```
@Modifying
@Query(value = "CALL virement(:src, :dst, CAST(:m AS NUMERIC))", nativeQuery = true)
void virement(@Param("src") int src, @Param("dst") int dst, @Param("m") BigDecimal m);
```

- Spring Data ne masque pas la base : il faut savoir lire le SQL qu'il produit (chapitres 1 à 3)

À retenir

- Repository = interface **métier** ; le service en dépend, jamais de JPA
- Spring Data génère l'implémentation : CRUD hérité, requêtes dérivées, `@Query`
- Projections pour ne lire que l'utile ; `Window` pour paginer sans `OFFSET`
- `@Transactional` sur le service, `@Version` (optimiste) ou `@Lock` (pessimiste) contre les mises à jour perdues
- `@EntityGraph` ou `JOIN FETCH` contre le N+1
- Toujours vérifier le SQL généré

Questions ?



Chouki.Tibermacine@univ-ubs.fr